

Chapitre 1 : Technologies de la programmation web

1. Définition

Les technologies de programmation web sont les outils et langages utilisés pour créer des sites et applications web. Elles évoluent constamment.

Technologies traditionnelles

- **HTML** : structure et contenu des pages.
- **CSS** : style et mise en forme.
- **JavaScript** : interactivité côté client ou serveur.
- **PHP, Java, Python, ASP.NET, Ruby on Rails** : développement côté serveur.
- **SQL** : gestion de bases de données relationnelles.

Technologies modernes

- **Front-end** : React, Angular, Vue.js, Sass, Webpack.
- **Back-end** : Node.js, Express.js, GraphQL, MongoDB, Firebase.
- **Déploiement** : Docker.

2. Développement web

- **Frontend** : visible par l'utilisateur.
 - Tâches : interface utilisateur, boutons, menus, formulaires, animations, responsive design.
 - Langages : HTML, CSS, JS.
 - Collaboration avec webdesigner et expert UX.
- **Backend** : côté serveur, non visible.
 - Tâches : logique serveur, API, sécurité, gestion des données.
 - Langages : PHP, Java, Python, C++.
 - Frameworks : Node.js, Laravel, Symfony, Django, Ruby on Rails.

3. Fonctionnement web

Monolithique (classique)

1. Navigateur envoie requête au serveur.
2. Serveur demande données à la base.
3. Serveur génère HTML complet et l'envoie au navigateur.

Séparation Front-end / Back-end (moderne, SPA)

1. Front-end envoie requête API (GET, POST...).
2. Backend récupère données de la base.
3. Backend renvoie réponse JSON.
4. Front-end construit l'interface avec les données reçues.

Avantages : rapide, flexible, compatible mobile, séparation claire des responsabilités.

4. API (Interface de Programmation)

- Permet la communication entre logiciels.
- **API Web / REST** : échanges via HTTP, réponses en JSON ou XML.
- Méthodes HTTP :
 - **GET** → lire
 - **POST** → créer
 - **PUT / PATCH** → mettre à jour
 - **DELETE** → supprimer

Parties d'une requête HTTP

Élément	Rôle
URL	Localise la ressource
Path Param	Identifie une ressource précise (/posts/1)
Query Params	Filtrer / trier / limiter (?limit=10)
Headers	Infos techniques (token, type de contenu...)
Body Params	Données envoyées au serveur (POST, PUT, PATCH)

5. Réponse HTTP

- **Statut** : code à 3 chiffres (200 OK, 404 Not Found, 500 Internal Server Error).
- **Headers** : infos serveur, type de contenu, cookies...
- **Body** : données renvoyées (HTML, JSON, XML...).

6. Outils

- **Postman / Insomnia** : tester et gérer les API.
- **Navigateurs (DevTools)** : inspecter requêtes HTTP et réponses.

Chapitre 2 : Programmation Asynchrone et Requêtes HTTP en JavaScript

1/Programmation Asynchrone

Synchronous vs Asynchronous

- **Synchrone (bloquant)** : les tâches s'exécutent une à une. Les opérations lentes bloquent tout le programme.
- **Asynchrone (non bloquant)** : les tâches s'exécutent indépendamment, le programme continue pendant que certaines opérations sont en cours.
 - Essentiel en JavaScript (monothread) pour éviter de bloquer l'interface (navigateur) ou le serveur (Node.js).

Exemples

- **Lecture de fichiers**
 - `fs.readFileSync` → bloquant
 - `fs.readFile` → non bloquant avec callback

Mécanismes de l'asynchronisme

1. **Callbacks** : fonctions passées en paramètre et exécutées après la fin d'une opération.
 - Exemple : `addEventListener`, `setTimeout`
 - Problème : imbriquer plusieurs callbacks → "callback hell"
2. **Promises** : objet représentant une valeur future ou une erreur.
 - `resolve` → succès, `reject` → échec
 - `.then()` → exécute après `resolve`
 - `.catch()` → exécute après `reject`
 - Permet de chaîner des opérations de manière lisible
3. **async/await** : syntaxe moderne qui rend le code asynchrone lisible comme du synchrone.
 - `async` → fonction asynchrone
 - `await` → attend la résolution d'une Promise
 - Avantages : code linéaire, facile à maintenir, évite le callback hell

Comparaison

Méthode	Lisibilité	Maintenance
Callbacks	Faible, imbriqué	Difficile

Promises	Moyenne, chaînage	Relativement facile mais répétitif
async/await	Très lisible	Très facile

2/Requêtes HTTP en JavaScript

Objectif

Permettre au client de communiquer avec un serveur (GET, POST, PUT/PATCH, DELETE) et traiter les réponses (souvent JSON).

Techniques

1. XMLHttpRequest (XHR / AJAX)

- Méthode ancienne, basée sur callbacks
- Permet GET, POST, PUT/PATCH, DELETE

Exemple :

```
let req = new XMLHttpRequest();

req.open('GET', url);

req.responseType = "json";

req.send();

req.onload = () => console.log(req.response);
```

- Avantages : compatible partout
- Limites : syntaxe lourde, moins lisible

2. Fetch API

- Méthode moderne basée sur Promises
- Syntaxe simple pour GET, POST, etc.

Exemple GET :

```
fetch(url)

.then(res => {

  if (!res.ok) throw new Error(res.status);

  return res.json();

})

.then(data => console.log(data))
```

```
.catch(err => console.error(err));
```

- Remarques :
 - `fetch()` ne rejette pas pour les erreurs HTTP (404, 500) → il faut vérifier `response.ok`
 - Rejette seulement en cas de problème réseau
- Avantages : lisible et chaînable avec `.then()`

3. Axios

- Bibliothèque externe (Promises)
- Syntaxe simple pour GET, POST, PUT, DELETE

Exemple GET :

```
axios.get(url)
```

```
.then(res => console.log(res.data))
```

```
.catch(err => console.error(err));
```

- Avantages :
 - Rejette automatiquement les erreurs HTTP (4xx, 5xx)
 - Données accessibles directement via `response.data`
 - Compatible navigateur et Node.js

Comparaison Fetch vs Axios

Critère	Fetch	Axios
Gestion erreurs HTTP	Manuelle (<code>response.ok</code>)	Automatique
Accès aux données	<code>response.json()</code> nécessaire	<code>response.data</code> direct
Simplicité	Moins simple	Plus simple, moins de code

Résumé global

- **Asynchronisme** : essentiel pour la réactivité, peut se faire via callbacks, promises ou `async/await`.
- **Promises & `async/await`** : code plus lisible et maintenable que les callbacks.
- **HTTP en JS** : XHR (ancien, callback), Fetch (promises natives), Axios (promises simplifiées et gestion automatique des erreurs).

Chapitre 3: Techniques de programmation coté serveur avec nodejs

1. Node.js

- **Définition** : Environnement d'exécution JavaScript côté serveur, pas un framework.
- **Fonction** : Exécute JS hors navigateur et fournit des bibliothèques pour applications serveur.
- **Moteur** : V8 de Chrome.
- **Avantages** :
 - Même langage front-end/back-end.
 - Très rapide (JIT).
 - Flexible via npm.
 - Polyvalent (serveur HTTP, API, streaming, desktop).
 - Large communauté et support.

2. Modèle d'exécution

- **Modèle classique (Apache/PHP)** : multithreading, bloquant, limité par le nombre de threads.
- **Node.js** : single-thread non bloquant, Event Loop.
 - Tâches rapides → exécutées immédiatement.
 - Tâches I/O → envoyées au thread pool (libuv).
 - Callback déclenché quand tâche terminée.

Avantages : léger, scalable, non bloquant.

Limites : CPU lourd ralentit Event Loop, asynchrone complexe.

3. Modules

- **Définition** : Fichier JS encapsulé (fonctions, classes, variables).
- **Types** :
 - **Core modules** (ex : fs, http, os)
 - **Modules npm** (installables via `npm install`)
 - **Modules personnalisés** (exports/imports)
- **Syntaxe import/export** :
 - CommonJS : `require()` / `exports.nom`
 - ES6 : `import` / `export`

4. npm et projets

- `npm init` / `npm init -y` → crée `package.json`.
- `package.json` : décrit projet, dépendances, scripts (`start`, `test`, etc.).

5. Serveur HTTP

- **Node.js pur** : `http.createServer()` + `server.listen(port)`
- **Routing simple** : analyse `req.url` pour renvoyer réponses différentes.

6. Express.js (micro-framework)

- Simplifie Node.js pur : routes, middlewares, templates.
- **Serveur de base** :

```
const express = require('express');
```

```
const app = express();
```

```
app.listen(3000);
```

Routes

- **Statique** : URL fixe, réponse identique.
- **Dynamique** : URL avec paramètres `:param`, accessibles via `req.params`.
- **Query parameters** : après `?` dans URL, accessibles via `req.query`.
- **Body parameters** : POST/PUT, JSON ou formulaire, nécessite `express.json()` / `express.urlencoded()`.
- **Headers** : `req.get('header')` ou `req.headers`.

Réponse

- `res.send()`, `res.json()`, `res.status()`, `res.redirect()`, `res.render()`, `res.sendFile()`.

7. Templates et SSR

- **EJS** : moteur de template Node.js.
- Permet variables `<%= %>`, conditions, boucles, inclusion de fichiers partiels.
- **Usage** : `res.render('template', data)` pour envoyer HTML dynamique.

8. Middlewares

- Fonction intercepte `req` et `res`, utilise `next()` pour continuer.
- **Types** :
 1. **Application** : `app.use(middleware)` → toutes les routes
 2. **Spécifique route** : `app.use('/route', middleware)`
 3. **Méthode HTTP spécifique** : `app.METHOD('/route', middleware, handler)`
 4. **Intégré** : `express.static()` pour fichiers statiques
 5. **Tiers** : Morgan, Helmet, CORS, etc.
 6. **Erreur** : 4 arguments (`err`, `req`, `res`, `next`) pour gérer erreurs et 404
 7. **Router** : regroupe routes dans modules séparés
- **Ordre d'exécution** : les middlewares s'exécutent dans l'ordre où ils sont déclarés.

9. Points clés

- Node.js → rapide, léger, scalable, asynchrone.
- Express → facilite routing, middlewares, templates, APIs REST.
- **req** contient params, query, body et headers.
- **res** permet d'envoyer réponses texte, JSON, fichiers ou templates.
- Middlewares → factorisent les traitements communs, gèrent erreurs et authentification.

Chapitre 4 : NodeJs et les bases de données

1. Types de bases de données

- **SQL / relationnelles** : données organisées en tables (lignes/colonnes) avec relations.
 - Exemples : MySQL, MariaDB, PostgreSQL, Oracle, SQL Server
 - Manipulation via **SQL** : CREATE, INSERT, SELECT, UPDATE, DELETE
- **NoSQL / non relationnelles** : données flexibles (documents, clés/valeurs, graphes).
 - Exemples : MongoDB, CouchDB, Redis, Cassandra
 - Manipulation via **ODM ou API spécifiques**

2. Outils pour gérer les bases

- **phpMyAdmin** : interface web pour MySQL/MariaDB
- **XAMPP** : serveur local (Apache + MySQL/MariaDB + PHP)

3. Pourquoi relationnelles et client-serveur ?

- Relations entre tables réduisent la redondance (ex. table Services séparée de table Employés)
- Modèle client-serveur : le serveur MySQL gère la base, les clients (Node.js, PHP...) envoient des requêtes SQL

4. SQL : catégories principales

- **LDD** : création/modification de tables (**CREATE TABLE**)
- **LMD** : manipulation des données (**INSERT, UPDATE, DELETE**)
- **LRD** : interrogation des données (**SELECT**)
- **LCT** : gestion des transactions (**COMMIT, ROLLBACK**)
- **LCD** : gestion des permissions (**GRANT, REVOKE**)

5. Node.js + MySQL

- Installer MySQL et modules Node.js : **mysql, express-myconnection**
- Configurer la connexion via middleware **express-myconnection**
- Utiliser **req.getConnection()** pour exécuter les requêtes SQL
- Exemple : récupération de données (**SELECT**) ou ajout depuis un formulaire (**INSERT**)

6. ORM et Sequelize (SQL via Node.js)

- **ORM** : manipule la BDD via des objets JS, simplifie CRUD
- **Sequelize** : ORM pour MySQL, MariaDB, PostgreSQL, etc.
- **Installation** : `npm install sequelize mariadb`
- **Connexion** : configurer Sequelize avec `dialect` et `host`
- **Modèles** : représentent les tables, définissent colonnes et types
- **CRUD avec modèles** : `create()`, `findAll()`, `findOne()`, `update()`, `destroy()`

7. MongoDB (NoSQL) et Node.js

- Base orientée documents, stocke des objets JSON (BSON) dans des **collections**
- Schéma flexible, documents différents dans une même collection
- Pas de relations automatiques comme SQL (références via `_id`)
- Node.js : utiliser le driver officiel `mongodb` ou **Mongoose** (ODM)
- **Mongoose** : crée un modèle pour chaque collection, fournit CRUD via objets JS

MAGUEMOUN SAMY